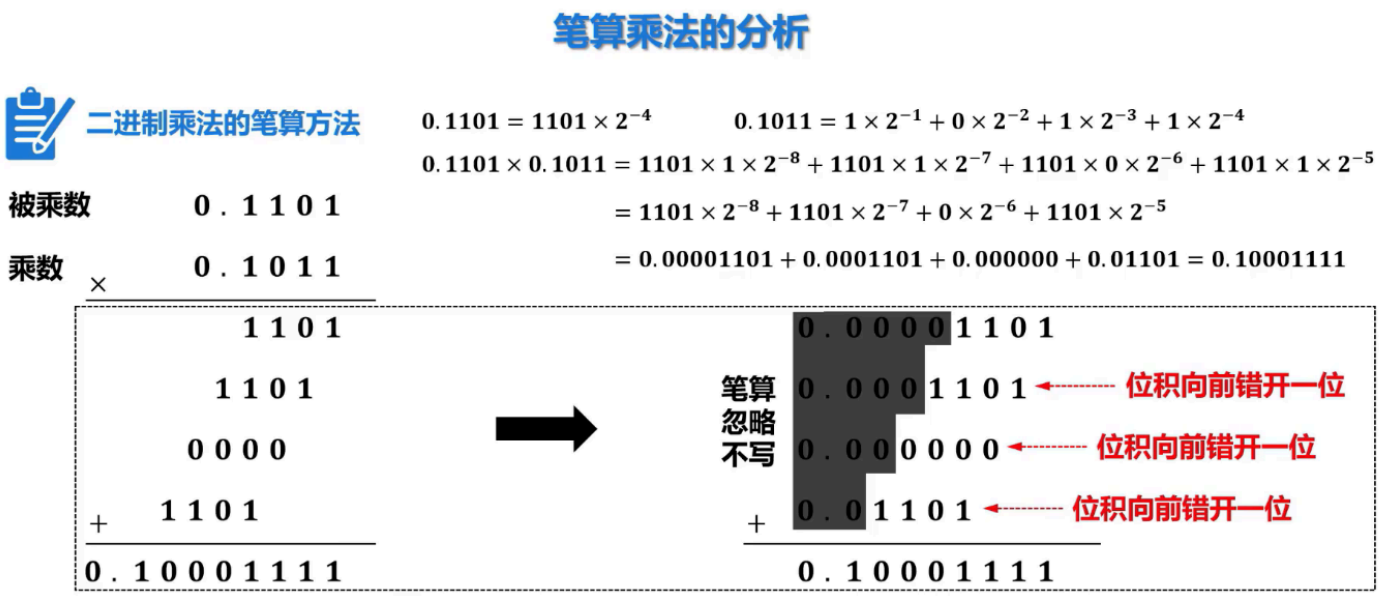
3.10 无符号数乘法的硬件逻辑实现

考纲定位：计算机组成原理 · 第 3 章 运算方法（3.10） | 重要度 ★★★★★ | 常见考法：选择题/大题（移位加、部分积、计数器、硬件流程）

一句话核心：乘法 = 移位 + 加法；从乘数最低位开始，逐位判断“加被乘数还是加 0”，再右移部分积

一、笔算二进制乘法



笔算

- 乘数每一位与被乘数相乘，得到部分积
- 部分积左移对应位权后相加
- 本质是“移位 + 加法”

二、机器实现

分析：乘法可写成加法，加法中的每一项又能写成科学计数法形式，相当于移位；所以机器实现需要加法与移位

2.1 传统笔算的劣势

机器完全模仿笔算乘法步骤，所遇到的问题

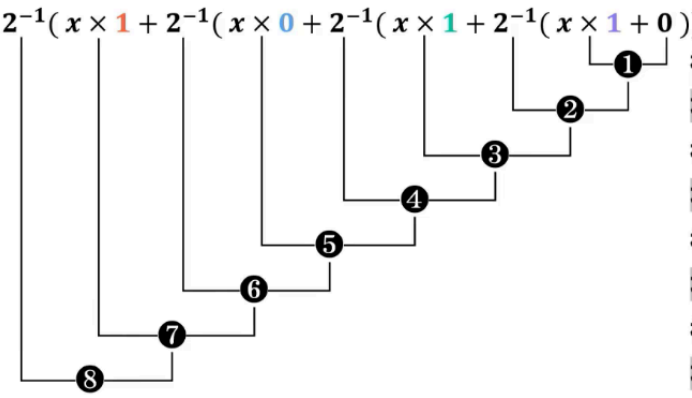
- **运算过程中，产生的多个位积需要使用多个寄存器进行暂存，以便最后进行一次性相加。**
- **乘积的数据宽度增长了一倍，这将造成硬件资源浪费和运算时间的增加。**

- 需要保存所有部分积，再统一相加
- 需要的寄存器多、加法器位数大
- 不适合硬件流水

2.2 优化后的方法

设 $x=0.1101$，$y=0.1011$，计算 $x \times y$。

$$x \times y = 2^{-1}(x \times 1 + 2^{-1}(x \times 0 + 2^{-1}(x \times 1 + 2^{-1}(x \times 1 + 0))))$$



被乘数 x 加 0
右移一位，得到 新的部分积
被乘数 x 加 部分积
右移一位，得到 新的部分积
被乘数 0 加 部分积
右移一位，得到 新的部分积
被乘数 x 加 部分积
右移一位，得到 最终乘积

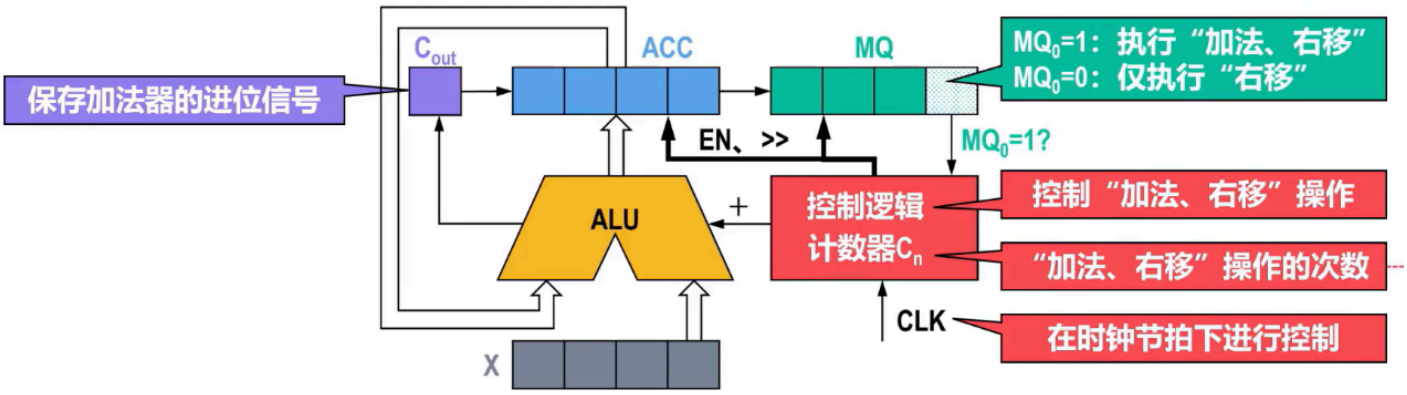
共4次
先加法，后移位
先加法，后移位
先加法，后移位
先加法，后移位

改进

- 从最低位开始，每一步先算“本位与上一个部分积的和”，然后再移位
- 只保留一个部分积，边算边移位
- 用计数器控制位数，循环 n 次

需要一个位宽为4位的寄存器存放被乘数x；还需要两个位宽为4位的寄存器分别存放部分积和最终乘积的高位和低位，初始阶段高位寄存器被清0，低位寄存器存放乘数y。再配上加法器及其他相应电路，即可实现乘法运算。

2.3 硬件逻辑实现



部件

- 被乘数寄存器 M
- 乘数寄存器 Q
- 部分积/累加器 A

- ALU：加法
- 计数器：控制循环次数

无符号乘法流程

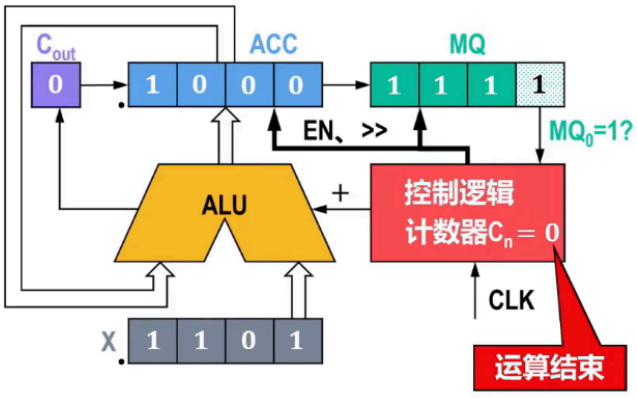
1. $A=0$，计数器 $=n$
2. 若 $Q_0=1$，$A \leftarrow A+M$；否则不加
3. A,Q 联合右移 1 位
4. 计数器减 1，不为 0 则重复
5. 结束后 A,Q 拼接为 $2n$ 位乘积

示例

设 $x=0.1101$，$y=0.1011$，计算 $x \times y$。

$$x \times y = 2^{-1}(x \times 1 + 2^{-1}(x \times 0 + 2^{-1}(x \times 1 + 2^{-1}(x \times 1 + 0)))) = 0.10001111$$

全部完成



	加数 被乘数	进位 信号	被加数、和 乘积高位	乘数 乘积低位
	X	C_{out}	ACC	MQ
初始状态	1101	0	0000	1011
(X) + (ACC) → ACC	1101	0	1101	1011
(C_{out})、(ACC)、(MQ) >> 1	1101	0	0110	1101
(X) + (ACC) → ACC	1101	1	0011	1101
(C_{out})、(ACC)、(MQ) >> 1	1101	0	1001	1110
(C_{out})、(ACC)、(MQ) >> 1	1101	0	0100	1111
(X) + (ACC) → ACC	1101	1	0001	1111
(C_{out})、(ACC)、(MQ) >> 1	1101	0	1000	1111

易错

- 移位是 A,Q 联合右移，不是只移 A
- 乘数最低位 Q_0 决定加不加
- 无符号乘法结果取 $2n$ 位，不能只取 n 位
- 计数器控制循环次数，不是由数据决定

知识关联：原码乘法（3.11 原码乘法的硬件逻辑实现）；Booth 算法（3.12 补码乘法的硬件逻辑实现——Booth算法）；阵列乘法器（3.13 无符号阵列乘法器）